

Semaine 1 | Module A

Fondamentaux du web & environnement

Anne Terrier — Développement web, classe passerelle

Cours 3h
TP 4h
Fil rouge MiniForum

- Comprendre le modèle client / serveur et le rôle des technologies web
- Distinguer HTML, CSS et JavaScript dans leurs rôles respectifs
- Distinguer un site statique d'un site dynamique
- Installer et configurer un environnement de développement fonctionnel
- Créer le dépôt Git de MiniForum avec un `.gitignore` correct dès le premier commit
- Connaître les règles du cours : fil rouge, jalons, usage de l'IA générative

1 Comment fonctionne le web ?

Quand tu tapes `https://www.hepia.ch` dans ton navigateur, une série d'échanges invisibles se produit en quelques millisecondes. Comprendre ce mécanisme est le point de départ de tout développeur web.

Aux origines du web

Année	Étape
1969	ARPAnet , ancêtre d'Internet, relie quatre universités américaines
1972	Apparition de l' e-mail
1989	Tim Berners-Lee rédige sa proposition au CERN , à Genève
1990	Premier serveur et premier navigateur web
1991	Le Web est ouvert au public
1993–1994	NCSA Mosaic puis Netscape Navigator font connaître le web au grand public

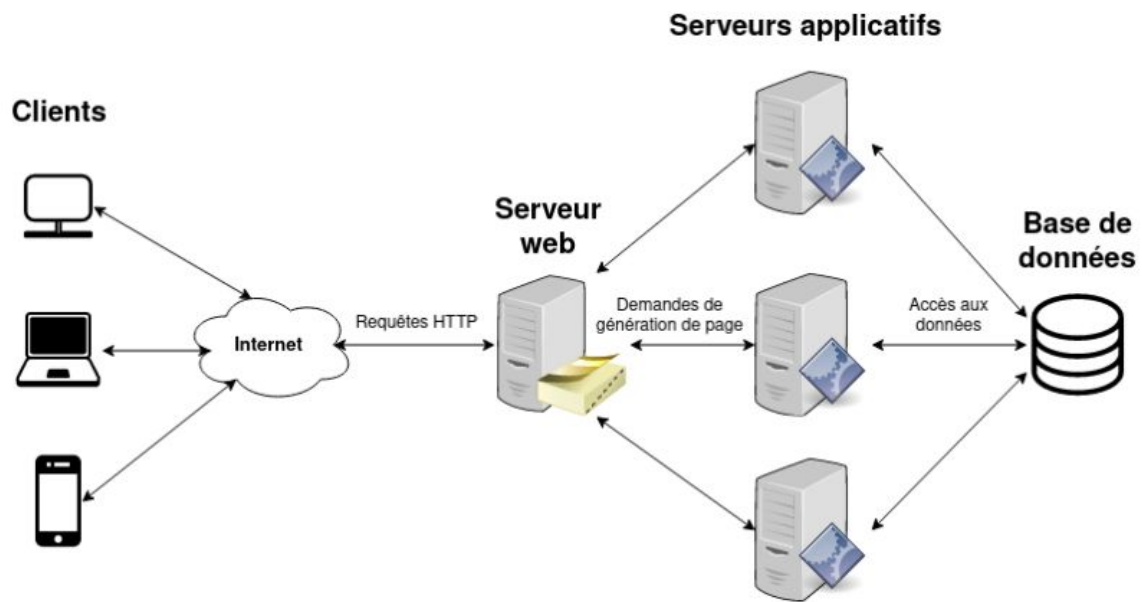
Deux confusions courantes

Internet n'est pas le Web. Internet est le réseau physique mondial qui relie les ordinateurs. Le Web n'est que l'un des services qui y circulent — l'ensemble des pages reliées par des **liens hypertexte** — au même titre que l'e-mail.

Un navigateur n'est pas un moteur de recherche. Le **navigateur** (Firefox, Chrome, Safari) traduit le code reçu pour afficher une page. Le **moteur de recherche** (Google, Bing, DuckDuckGo) est un site web qui aide à trouver des adresses. Google Chrome est un navigateur ; Google est un moteur de recherche.

Chaque navigateur interprète le code à sa manière, et l'affichage peut varier de l'un à l'autre. D'où une habitude à prendre dès maintenant : **tester ses pages sur au moins deux navigateurs différents.**

Le modèle client / serveur



Client : programme qui *demande* une ressource (le navigateur).

Serveur : programme qui *répond* à la demande en envoyant des fichiers ou des données.

La communication se fait via le protocole **HTTP** (ou **HTTPS** pour la version chiffrée).

Anatomie d'une URL

```
https://www.hepia.ch/formations/informatique
```

↑
Protocole

↑
Domaine

↑
Chemin

DNS — du nom à l'adresse

Un ordinateur ne comprend pas `www.hepia.ch` : il a besoin d'une adresse numérique, appelée **adresse IP** (par exemple `192.0.2.34`). Le **DNS** (*Domain Name System*) est le service qui traduit un nom de domaine lisible en adresse IP.

Le DNS fonctionne comme un annuaire : tu connais le *nom* d'une personne (le domaine), l'annuaire te donne son *numéro* (l'adresse IP), et c'est ce numéro qui permet réellement d'établir la communication.

Étapes simplifiées :

1. Le navigateur demande au DNS : « quelle est l'adresse IP de `www.hepia.ch` ? »
2. Le serveur DNS répond avec l'adresse IP correspondante
3. Le navigateur peut alors envoyer sa requête HTTP directement à cette adresse

Les codes de statut HTTP

Chaque réponse du serveur commence par un nombre à trois chiffres qui indique comment la requête s'est passée. Le premier chiffre donne la famille : 2xx succès, 3xx redirection, 4xx erreur du client, 5xx erreur du serveur.

Code	Signification	Quand ?
200	OK	Tout s'est bien passé
301	Moved Permanently	La ressource a changé d'adresse
403	Forbidden	L'accès est refusé
404	Not Found	La ressource n'existe pas
500	Internal Server Error	Erreur côté serveur

Ces codes reviendront en semaine 13, lorsque tu écriras toi-même le serveur qui les renvoie. Pour l'instant, il suffit de savoir les lire dans l'onglet *Réseau* des outils de développement du navigateur.

Sites statiques et sites dynamiques

Un **site statique** renvoie toujours le même contenu au client : le fichier HTML existe déjà sur le serveur et est envoyé tel quel.

Un **site dynamique** génère le contenu à la demande, souvent à partir d'une base de données : deux visiteurs, ou deux moments différents, peuvent recevoir des pages différentes.

	Site statique	Site dynamique
Contenu	Identique pour tout le monde	Généré à chaque requête
Exemple	Page vitrine, portfolio	Forum, boutique en ligne, réseau social
Techniquement	Fichiers HTML / CSS / JS seuls	Serveur applicatif (Node.js) + base de données

Où se situe MiniForum ?

Le projet fil rouge **MiniForum** passe par trois états successifs :

- **semaines 1 à 5** — site **statique** : contenu écrit en dur dans le HTML ;
- **semaines 6 et 7** — site **interactif mais volatile** : JavaScript manipule des données stockées en mémoire, qui disparaissent à chaque rafraîchissement ;
- **à partir de la semaine 12** — site **dynamique** : Node.js / Express interroge une vraie base de données MySQL.

Cette frustration de la semaine 7 est voulue : c'est elle qui rend évidente la nécessité d'une base de données.

2 Les trois langages côté client



Les murs

La peinture

L'électricité

Analogie : la construction d'une maison

Important — Qui exécute quoi ?

HTML, CSS et JavaScript s'exécutent tous les trois dans le **navigateur**, donc côté client. Le serveur se contente de les envoyer ; il ne les interprète pas.

Un premier exemple

```
1 <!DOCTYPE html>
2 <html lang="fr">
3 <head>
4   <meta charset="UTF-8">
5   <title>MiniForum</title>
6   <link rel="stylesheet" href="style.css">
7 </head>
8 <body>
9   <h1>Bienvenue sur MiniForum</h1>
10  <button id="btn">Cliquez-moi</button>
11  <script src="app.js"></script>
12 </body>
13 </html>
```

Extrait 1 – index.html — la structure

```
1 button {
2   background-color: #1D4E89; /* bleu cfpt.i */
3   color: white;
4   padding: 10px 20px;
5   border: none;
6   border-radius: 4px;
7   cursor: pointer;
8 }
```

Extrait 2 – style.css — la présentation

```
1 // On récupère le bouton, puis on réagit au clic
2 const btn = document.getElementById('btn');
3
4 btn.addEventListener('click', () => {
5   alert('Bonjour depuis JavaScript !');
6 });
```

Extrait 3 – app.js — le comportement

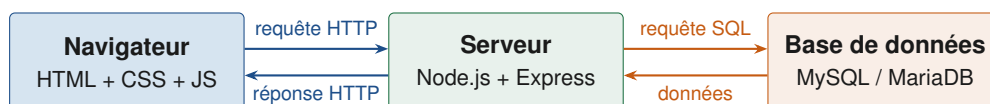
La balise `<script>` est placée **juste avant** `</body>`, donc après le bouton. Si elle était dans le `<head>`, le script chercherait un bouton qui n'existe pas encore et `getElementById` renverrait `null`. C'est l'une des erreurs les plus fréquentes en début d'apprentissage.

3 Les langages côté serveur

Jusqu'ici, tout se passait dans le navigateur. Mais une application web doit aussi enregistrer et retrouver des données, ce que le navigateur seul ne sait pas faire durablement. Pour cela, il dialogue avec un programme exécuté sur un serveur.

Un langage côté serveur s'exécute **sur le serveur**. Il reçoit une requête du navigateur, la traite — souvent en interrogeant une base de données — puis renvoie une réponse.

La chaîne complète



À noter dès maintenant

Le navigateur ne parle **jamais** directement à la base de données. Il passe toujours par le serveur. Cette règle paraît anodine ; elle est en réalité au cœur de la sécurité d'une application web, et on y reviendra en semaine 19.

Quelques langages côté serveur

Contrairement au navigateur, qui n'exécute que du JavaScript, un serveur n'est pas limité à un seul langage.

Langage	Exemple d'utilisation
PHP	Sites et applications web (WordPress, Symfony)
Python	Applications web et API (Django, Flask)
Java	Applications d'entreprise (Spring)
C#	Applications web avec ASP.NET
JavaScript avec Node.js	Serveurs web et API – notre choix

Node.js

Dans ce cours, nous utiliserons **Node.js**, qui permet d'exécuter du JavaScript en dehors du navigateur. Il permet notamment de :

- créer un serveur web ;
- recevoir et traiter des requêtes HTTP ;
- communiquer avec une base de données ;

- exposer une API.

Important — Le même langage, deux environnements

JavaScript côté client s'exécute dans le navigateur : il a accès à la page (le DOM), pas au disque dur.

JavaScript côté serveur s'exécute avec Node.js : il a accès au système de fichiers et à la base de données, mais pas à la page.

Même langage, mêmes règles de syntaxe, mais deux mondes qu'il ne faut pas confondre.

4 La stack du cours

Dans le projet **MiniForum**, chaque technologie occupe une place précise.

HTML5 + CSS

Structure & présentation — semaines 2 à 5

JavaScript ES6+

Interactivité dans le navigateur — semaines 6 et 7

Node.js + Express

Serveur HTTP & API REST — semaines 12 à 17

MySQL / MariaDB

Base de données relationnelle — semaines 8 à 11

Git + GitHub

Gestion de version — de la semaine 1 à la semaine 20

Côté client

Côté serveur

Une application web complète comporte presque toujours trois parties : une **interface** dans le navigateur, un **serveur** qui traite les demandes, une **base de données** qui conserve l'information.

La particularité de ce cours : nous utilisons **JavaScript des deux côtés**. Un seul langage à apprendre au lieu de deux.

5 Environnement de développement

Outils à installer

- | | |
|-----------------------------|------------------------------------|
| • Visual Studio Code | <code>code.visualstudio.com</code> |
| • Node.js 24 LTS | <code>nodejs.org</code> |
| • Git | <code>git-scm.com</code> |
| • Un compte GitHub | <code>github.com</code> |

Version de Node.js

Installe bien la version **24 LTS**. Node.js 20, encore proposée sur beaucoup de tutoriels, est en fin de vie et ne doit plus être utilisée. En cas de doute, `node -v` doit afficher un numéro commençant par v24.

Extensions VS Code

Extension	Utilité	Dès
Prettier	Formatage automatique du code	S1
ESLint	Détection d'erreurs JavaScript	S1
Live Server	Rechargement automatique du navigateur	S1
GitLens	Visualiser l'historique Git dans l'éditeur	S1
REST Client	Tester les routes d'une API depuis VS Code	S13
SQLTools	Se connecter à une base MySQL depuis VS Code	S8

À désinstaller : les extensions d'assistance IA

GitHub Copilot et toute extension équivalente doivent être **désinstallées** de Visual Studio Code cette semaine. Leur absence est vérifiée à chaque jalon. Les raisons sont détaillées en section 7.

Premiers pas dans le terminal

Git et Node.js s'utilisent en ligne de commande. Quelques commandes suffisent pour naviguer dans les dossiers de ton projet.

Commande	Effet
<code>pwd</code>	Affiche le dossier courant
<code>ls (ou <code>dir</code>)</code>	Liste le contenu du dossier courant
<code>cd nom-dossier</code>	Se déplace dans un sous-dossier
<code>cd ..</code>	Remonte d'un niveau
<code>mkdir nom</code>	Crée un nouveau dossier
<code>touch nom.txt</code>	Crée un fichier vide (Git Bash, macOS, Linux)

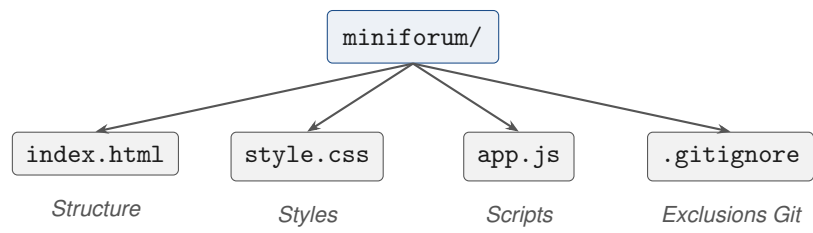
Ces commandes sont revues et approfondies dans le module Linux / Bash donné en parallèle. Il suffit ici de savoir se déplacer et créer un dossier pour démarrer MiniForum.

MDN, le bon réflexe documentation

MDN Web Docs (developer.mozilla.org) est la référence pour HTML, CSS et JavaScript. C'est la première source à consulter — avant un forum, avant une vidéo.

Tape dans ton moteur de recherche `mdn` suivi du nom de l'élément cherché : `mdn addEventListener`, `mdn input type`, `mdn flexbox`. Sur une page MDN, trois zones sont utiles : la **description**, la **syntaxe** et surtout les **exemples**, qui sont modifiables directement dans la page.

Structure du projet MiniForum



6 Git — premiers pas

Git est un outil de **gestion de version** : il enregistre l'historique de chaque modification de ton code, permet de revenir en arrière en cas d'erreur, et facilite le travail en équipe. C'est un outil professionnel incontournable, utilisé dans pratiquement tous les projets informatiques.

Pourquoi utiliser Git ?

Sans Git, on se retrouve rapidement dans cette situation :

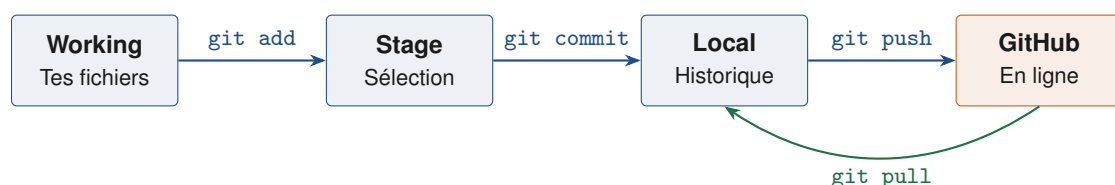
Fichier	Ce que tu essayais de faire
index.html	Version qui marche
index_final.html	Tentative d'amélioration
index_final2.html	Autre tentative
index_VRAI_final.html	On ne sait plus...

Git résout ce problème : un seul fichier, un historique complet, la possibilité de revenir à n'importe quel état passé.

- **Sauvegarder** l'historique complet de son projet, modification par modification
- **Revenir en arrière** à n'importe quel état antérieur du code
- **Travailler en équipe** sans écraser le travail des autres
- **Expérimenter** sans risque, sur une branche séparée
- **Synchroniser** son travail entre plusieurs machines via GitHub

Les quatre zones de Git

Git organise le travail en quatre zones distinctes. Les comprendre évite bien des blocages face aux commandes.



Zone	Description
Working Directory	Tes fichiers tels que tu les vois dans ton éditeur. Toutes tes modifications vivent ici.
Stage (Index)	Zone de préparation : tu sélectionnes les fichiers à inclure dans le prochain commit.
Dépôt local	L'historique Git stocké sur ta machine. Chaque commit est un instantané du projet.
Dépôt distant (GitHub)	Copie du dépôt hébergée en ligne. Permet la sauvegarde et le partage.

Working Directory : tout le monde est dans la salle. **Stage** : tu demandes à certaines personnes de se mettre en position. **Commit** : tu prends la photo — c'est un instantané permanent. **Push** : tu envoies la photo sur un serveur partagé pour que tout le monde y accède.

Commandes essentielles

```
1 # Initialiser un dépôt Git dans le dossier courant
2 git init
3
4 # Voir l'état des fichiers (modifiés, en attente, non suivis)
5 git status
6
7 # Ajouter tous les fichiers modifiés au stage
8 git add .
9
10 # Ajouter un seul fichier
11 git add index.html
12
13 # Enregistrer un commit avec un message
14 git commit -m "Structure de base du MiniForum"
15
16 # Voir l'historique des commits
17 git log --oneline
```

Extrait 4 – Initialiser un projet Git

```
1 # Lier le dépôt local à un dépôt GitHub
2 git remote add origin https://github.com/ton-login/miniforum.git
3
4 # Envoyer les commits sur GitHub (la première fois)
5 git push -u origin main
6
7 # Envoyer les commits suivants
8 git push
9
10 # Récupérer les modifications depuis GitHub
11 git pull
```

Extrait 5 – Synchroniser avec GitHub

```
1 # Voir les modifications faites sur un fichier
2 git diff index.html
3
4 # Annuler les modifications d'un fichier (avant git add)
5 git restore index.html
6
7 # Voir le détail d'un commit
8 git show abc1234
```

Extrait 6 – Revenir en arrière

Un bon message décrit **ce qui a changé**, pas comment.

OUI Ajout du formulaire de création de sujet

OUI Correction de l'affichage de la liste des messages

NON modifier **NON** ça marche **NON** update

Le fichier .gitignore

Certains fichiers ne doivent jamais être versionnés : mots de passe, dépendances téléchargées, fichiers temporaires du système. Le fichier .gitignore, placé à la racine du dépôt, indique à Git ce qu'il doit

ignorer.

```
1 # Dépendances Node.js : jamais dans Git
2 node_modules/
3
4 # Variables d'environnement (mots de passe, clés d'API)
5 .env
6
7 # Fichiers système
8 .DS_Store
9 Thumbs.db
10
11 # Réglages locaux de l'éditeur
12 .vscode/
```

Extrait 7 – .gitignore — version à créer dès la semaine 1

Important — Le .gitignore vient avant le premier commit

Le dossier `node_modules/` peut contenir des milliers de fichiers et peser plusieurs centaines de mégaoctets ; les dépendances se réinstallent en une commande (`npm install`) et n'ont donc pas leur place dans l'historique.

Surtout : un fichier déjà commité reste dans l'historique **même si on l'ajoute au .gitignore ensuite**. C'est la raison pour laquelle on crée ce fichier **avant** le tout premier commit — et c'est aussi ce qui évite, plus tard dans l'année, de publier par erreur un mot de passe de base de données sur GitHub.

7 Le cours : fil rouge, jalons et règles

MiniForum, le projet fil rouge

Plutôt que des exercices isolés, l'ensemble du cours est construit autour d'un projet unique qui grandit de semaine en semaine : **MiniForum**, un forum simplifié avec des utilisateurs, des sujets de discussion, des messages et des réactions.

Phase	Sem.	Ce que tu construis	Jalon
1	1–7	HTML / CSS / JS : une interface interactive, données en mémoire	J1 (S7)
2	8–11	SQL : modélisation et requêtes sur une vraie base	J2 (S11)
3	12–17	Node.js / Express : une API REST branchée sur MySQL	J3 (S17)
4	18–20	Authentification, sécurité, soutenance	Soutenance

Comment le cours est évalué

Épreuve	Objet	Sem.	Poids
Jalon 1	Interface MiniForum interactive	7	10 %
Jalon 2	Modèle de données et requêtes SQL	11	10 %
Contrôle écrit	Individuel, sur machine, sans réseau	12	25 %
Jalon 3	API Express connectée à MySQL	17	10 %
Projet final	MiniForum complet	20	30 %
Soutenance	Présentation orale et démonstration	20	15 %

Un projet mené sur vingt semaines a un défaut : rester bloqué en semaine 4 pourrait signifier ne plus jamais rattraper le retard.

Pour l'éviter, un **dépôt de référence** est fourni à la fin de chaque phase (semaines 7, 11 et 17). Il contient un MiniForum conforme à l'état attendu. Chacun peut repartir de ce dépôt au début de la phase suivante, **sans pénalité** sur les jalons ultérieurs. Un retard ne doit jamais se transformer en abandon : il faut simplement le signaler.

Usage de l'intelligence artificielle générative

Les assistants de génération de code sont interdits dans ce cours

GitHub Copilot, ChatGPT, Claude, Gemini et tout autre assistant produisant du code sont **interdits** pour l'ensemble des travaux rendus : travaux pratiques, jalons, projet final et contrôle écrit.

Pourquoi. L'objectif de ce cours est que des personnes n'ayant jamais programmé sachent programmer à la fin de l'année. Cette compétence s'acquiert en écrivant du code, en le voyant échouer et en comprenant pourquoi. Un assistant qui produit immédiatement une réponse correcte supprime exactement l'étape où l'apprentissage se produit. L'enjeu est concret : en 1^{re} année hepia, les évaluations d'algorithmique sont individuelles et sans assistance.

Autorisé	Condition
MDN, documentation Express, manuel MySQL	Aucune, c'est même recommandé
Moteurs de recherche, Stack Overflow	Comprendre et savoir expliquer tout code repris
Entraide entre étudiant-e-s	Chacun-e écrit son propre code
Prettier, ESLint, correcteur orthographique	Ce sont des outils de mise en forme, pas de génération

Comment c'est vérifié. Les extensions d'assistance IA sont désinstallées en semaine 1 et leur absence est contrôlée aux jalons. L'historique Git est examiné : un projet qui apparaît en un seul commit massif donne lieu à un entretien. Chaque jalon comporte des questions orales sur le code rendu, et le contrôle écrit de la semaine 12 se déroule sur machine sans accès au réseau. Le manquement à cette règle relève du règlement de l'établissement en matière de fraude.

Cette interdiction porte sur la **période d'apprentissage des fondamentaux**, pas sur la pratique professionnelle. Les usages en contexte professionnel sont abordés en semaine 20, dans le bilan de fin de cours.

8 Lexique de la semaine

Terme	Définition
URL	Adresse permettant de localiser une ressource sur le web
Adresse IP	Identifiant numérique d'une machine sur un réseau
DNS	Service qui traduit un nom de domaine en adresse IP
HTTP / HTTPS	Protocole de communication entre client et serveur (HTTPS = chiffré)
Code de statut	Nombre à trois chiffres résumant l'issue d'une requête (200, 404...)
Client	Programme qui demande une ressource (le navigateur)
Serveur	Programme qui répond en fournissant fichiers ou données
Statique	Contenu identique quel que soit le visiteur ou le moment
Dynamique	Contenu généré à la demande, souvent depuis une base de données
Dépôt (<i>repository</i>)	Dossier suivi par Git, contenant l'historique du projet
Commit	Instantané enregistré de l'état du projet
.gitignore	Fichier listant ce que Git doit ignorer

Résumé de la semaine

- Le web repose sur un modèle **client / serveur** et sur le protocole **HTTP**
- Le **DNS** traduit un nom de domaine en adresse IP
- **HTML** structure, **CSS** présente, **JavaScript** rend interactif — les trois s'exécutent dans le navigateur
- Un site **statique** renvoie toujours le même contenu ; un site **dynamique** le génère à la demande
- **Node.js** permet d'exécuter du JavaScript côté serveur : un seul langage pour toute la stack
- **Git** : add → commit → push, avec un **.gitignore** dès le premier commit
- **MiniForum** est le projet fil rouge, développé sur les **20 semaines** du cours

Pour la semaine 2

- VS Code, Node.js 24 et Git installés et vérifiés (`node -v`, `git -version`)
- Extensions d'assistance IA désinstallées de VS Code
- Compte GitHub créé
- Dépôt miniforum poussé sur GitHub, avec `.gitignore` dans le premier commit

Le détail pas à pas se trouve dans la fiche *S01_TP*.